

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00174351 - B00174492
Student Name:	Benjamin Oludipe – Samuel Sebastine

Module Information

Module Title:	Scripting for CyberSecurity
Module Code:	H2702

Assessment Information

Assessment Title:	CA3
-------------------	-----

Declaration

YES ☒ I confirm that I have read and understood the University's Academic Integrity Policy.

<https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/>

YES ☒ I have completed the TU Dublin AI Driving License theory test.

<https://www.tudublin.ie/explore/about-the-university/academic-affairs/ita/ita-resources/artificial-intelligence-driving-license/>

YES ☒ I declare that this assessment submission is my own work.

YES ☒ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Benjamin.Oludipe Samuel Sebastine
Date:	14/12/25

introduction

This project involves design and implementation of a small cybersecurity automation tool that uses powershell and python to support analysis of suspicious network activity. The specific focus of the project was to identify external IP addresses that a system is currently communicating to bolster make this information using external threat intelligence. This type of task is common in security operations, where analysts need to determine whether observed network connections may be seen as a security risk.

The implemented solution automates a workflow that would require several manual steps. First, active network connections are inspected, and remote IP addresses are taken out using a powershell script. These IP addresses are then checked against the AbuseIPDB threat intelligence service using a python script. The final output is structured json report that provides good information for each observed remote IP address.

The goal of this project was to show practical scripting skills, use of APIs, and ability to use multiple scripting languages into a unified cybersecurity workflow. The tool was designed to run on macos using powershell core and python 3, making sure its compatible with the development environment while remaining aligned with concepts covered in the lectures.

Tool design

The project was designed as two-stage automation pipeline, with each stage integrated using the scripting language best suited to the task. powershell was used for collecting system level network artefacts, while python was used for processing the collected data, interacting with an external security api, and generating the module lectures.

Powershell component

The powershell scripts is responsible for extracting remote IP addresses from active tcp connections on the system. powershell core was chosen for this task due to its strong support for pipelines, text processing, and structured objects, all of which were covered in the module lectures.

The script uses netstat command to list active network connections and filters the output to include only connections in an established state. This ensures that the script focuses on active communication rather than closed or listening connections. From each established connection, the remote IP address is extracted and processed.

To improve the quality of the output, the script removes duplicate IP addresses and converts the results into custom PowerShell objects. These objects are then exported to a csv file named `remote_ips.csv`. This file serves as a clean and lightweight input for the python component of the workflow. The use of csv allows the data to be easily shared and processed across scripting languages.

Python component

The python script performs threat intelligence enrichment stage of the workflow. Python was selected due to its sustainability for api interaction, availability of third-party libraries, and ease of working with structured data formats such as json.

The script begins by reading the configuration from a json file. This file contains the api key required to authenticate with the abuseIPDB service. Separating configuration data from source code reflects good security practice and makes the script easier to maintain.

The script then reads the list of remote IP addresses from the csv file generated by the powershell script. Using loops and conditional logic, each IP address is processed individually. For each IP, the script sends a request to the abuseIPDB api using the requests library. The api response is parsed to extract relevant reputation information, such as the abuse confidence score and the associated country.

Error handling is implemented using `try` and `except` blocks to ensure that issues such as network errors or unexpected API responses do not cause the script to fail. All results are stored in a python data structure and written to a json file named `threat_report.json`, which represents the final output of the tool.

Cybersecurity context

When we analysed outbound network connections we seen its a fundamental task in cybersecurity, systems that are compromised by malware attacks more than likely communicate with external servers for control purposes. as a result, identifying and seeing external ip addresses involved in network communication is and something important when it comes to incident response.

Threat intelligence services such abuseIPDB which we used many times in our labs, which was a good external context by seeing reports of abusive or malicious behavior associated with ip addresses. By seeing network activity with threat intelligence data, analysts can check investigations and focus on connections that are more likely to cause a security risk.

The tool developed in this project mirrors real world workflow, by automating the extraction of remote ip addresses and enhancing them with reputation data, the tool reduces the time and effort required for initial analysis.

Tool demonstration

The tool is executed as a simple two-step process, the powershell script is run using powershell core on macos. When executed the script inspects active tcp connections, extracts remote ip addresses, and exports the results to the remote_ips.csv file. The file then provides a list of external systems that the host is actively communicating with. After this the python script is executed. This script reads the api key from the configuration file and processes each Ip address listed in the csv file. For each Ip, a request is sent to the abuseIPDB api, and the returned reputation data is analysed. The results of this analysis are written to the threat_report.json file.

The successful generation of the json report demonstrates that the combined workflow functions correctly from data collection through to threat intelligence. The output file can be reviewed by an analyst to quickly assess whether any of the observed network connections may be associated with known malicious activity.

Reflection

This project was completed as a group assignment by, me and Sam. We worked together throughout the project, from the initial planning and design stages to implementation, testing, and final documentation.

The overall workflow, tool design, and cybersecurity use case were agreed upon jointly before development began.

I focused primarily on the powershell component of the project. This includes developing the script used to extract remote IP addresses from active network connections, implementing filtering and logic, and putting the results to a csv file. I also contributed to testing the powershell script on macos using powershell core and ensuring that the output was suitable for further processing by the python script.

Sam focused primarily on the python component and the integration of the abuseIPDB threat intelligence service. This involved reading input data from the csv file, parsing configuration information from a json file, querying the abuseIPDB api using an external library, and generating the final json report. Sam also worked on handling api-related errors and verifying the structure and usefulness of the output for a security analyst.

We worked together on the powershell and python components into a workflow, debugging issues related to file handling and configuration, and preparing the readme and technical report. One of the main challenges faced during the project was ensuring smooth execution on macos and resolving configuration issues related to json and api keys. Through this work, we both gained experience in scripting, automation, and applying threat intelligence to a realistic cybersecurity task.